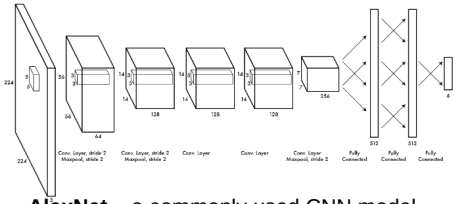
Reducing the Memory Footprint of Fast Convolution

Shiva K Deviah, Tze Meng Low, and Franz Franchetti

Issue

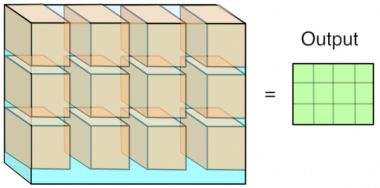
Caffe is the de facto standard framework for building arbitrarily complex machine learning models, in particular, Convolutional Neural Networks (CNNs). Caffe is hugely popular simply because it can compute convolution fast. But greater speed comes at a price… and that price is **EXTRA MEMORY.**



AlexNet – a commonly used CNN model

Textbook method convolution

Convolve filter over image to get your output – repeat for all filters

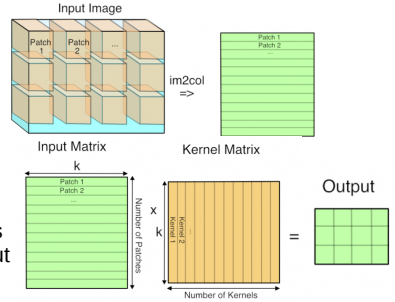


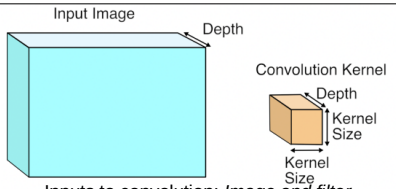
VS

Caffe's method of convolution

Step 1
reshape image (flatten out local regions)

Step 2
matrix multiply image with filters to get your output





Inputs to convolution: *Image and filter*

How is memory the issue?

- Convolution requires reuse of data
- Flattening out local regions leads to wasteful replication of data
- Increased space required for flattening

Solution

Below, we present ***DietConv****, an algorithm that strives to achieve the same speed as Caffe without "consuming" as much memory:*

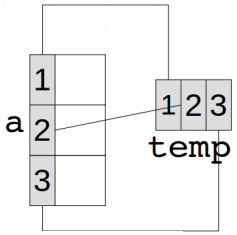
Inputs: **a** (**`CxIxI`** image) and **w** (**`KxCxFxF`** filter)
Output: **r** (**`KxRxR`** output where
`R = (I – F) / stride + 1)`

Step 1: Reshape filter
From a **`KxCxFxF`** matrix to a **`FxKxCxF`** matrix. This also involves reversing the filters appropriately so correlation can be carried out on reversed filters.

Step 2: Run algorithm

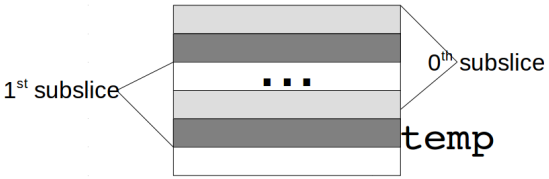
```
for i ← 0 to R do
    temp = ith slice of a
    for j ← 0 to F do
        r[i] += dot(w[j], jth sub-slice of temp)
        /* KxCxF . CxFxR = KxR */
        end
end
```

Step 3: Reshape result
From an **`RxKxR`** matrix to a **`KxRxR`** matrix.



(Left) Each pass, stride right by **`stride`** and copy the data into **`temp`**. This is necessary because **`cblas_sgemm`** cannot read the data directly

(Below) One section of **`temp`** will supply **`F`** different slices of data for convolution

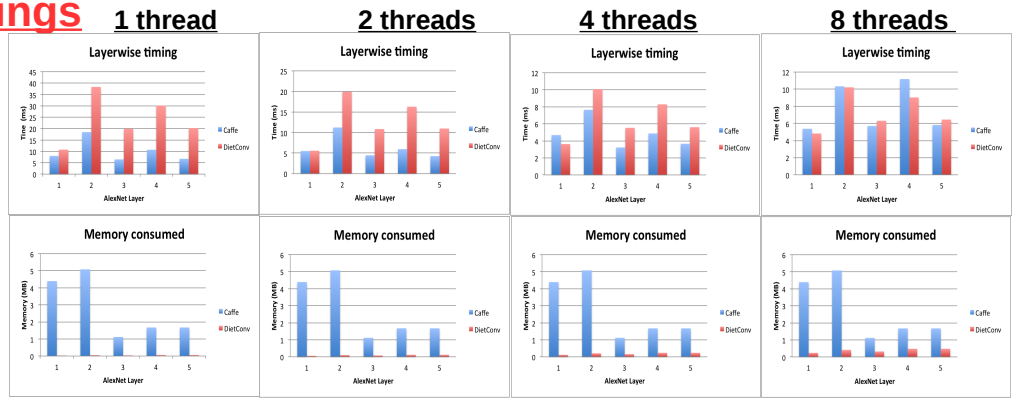


By judiciously applying parallelism on the outer loop as well as the copying of **a** into **`temp`**, we can achieve considerable speedups, as shown in the next section.

AlexNet Convolution Timings

(benchmarked on Ubuntu OS *with* Intel Xeon @ 3.30GHz *x* 8 *cores)*

- Caffe uses parallelised OpenBLAS whereas ***DietConv*** has serial OpenBLAS with OpenMP to parallelize the outer loops and copying operation
- Peak performance is achieved at 4 threads
- ***DietConv*** uses upto 150x lesser memory than Caffe with a single thread.



Future Enhancements

This method of convolution is far from perfect since it still relies on extra memory and some amount of preprocessing. As further enhancements, we aim to:

- improve performance for lower thread count
- reduce memory consumption; so it is not affected by thread count
- explore other avenues of computing intermediate sums and products that can hopefully lead to faster computation
- integrate ***DietConv*** into existing CNN frameworks to enable faster training.